Chapter 11: Publishing Your Application

When you build Windows desktop applications, there's another stage of the project that is equally important to the development phase: the deployment. If you build the best application in the world, but the deployment process is complicated and prone to errors, your customers will be equally dissatisfied, and they will just give up.

However, the deployment technology is only one side of the coin: once you have found the right one, you need a platform to distribute it.

In this chapter, we're going to address all these challenges by going through the following topics:

- Understanding the MSIX packaging technology
- Creating and signing an MSIX package
- Publishing your application to Microsoft Store
- Publishing your application to a website using AppInstaller
- Publishing your application to the Windows Package Manager repository

We have mentioned MSIX and the Windows App SDK package model multiple times in this book. It's time to dive a little bit deeper into the topic and gain a better understanding of what MSIX can do for us.

Understanding the MSIX packaging technology

MSIX is a new packaging technology introduced by Microsoft with the goal to solve many of the challenges that the previous installation technologies (such as MSI or ClickOnce) have created over the years. Some of these challenges include the following:

- The need for enterprises to repackage applications at every update when they have to make any customization to satisfy the company's requirements
- The requirement of using third-party tools or Visual Studio customizations to create an installer as part of the build process
- The need to use third-party services (or build your own custom one) if you want to provide automatic updates
- The inability to completely uninstall an application often leaves orphan registry keys or files, which, over time, slow down the operating system

The key difference between MSIX and other technologies is that it's completely integrated into Windows. This means that many of the challenges that we have outlined are solved without requiring any extra work from the developer. Let's briefly look at all of the main advantages of this technology.

Built-in optimizations around network bandwidth and disk space

Thanks to the Windows integration, MSIX can provide two features that are especially helpful in enterprise environments, where applications are deployed across thousands of devices scattered across different locations in the world.

The first one is differential updates. Whenever you release an update of your application, as a developer, you just need to create a new MSIX package with all of the content. The package can either be used for the first installation or to update an existing installation. In the second scenario, Windows won't download the entire content of the package,

but only the files that have changed from the previous version. This feature introduces advantages for both users and developers:

- Users will save time and bandwidth since they won't have to re-download the entire application from scratch.
- Developers don't need to create a separate packaging and deployment process for the main application and updates. They can just create a single MSIX package that is optimized for both scenarios.

The second optimization is around disk space. If you install an MSIX package that contains a file, with the same hash of another file included in another MSIX package, Windows won't download it again and won't store a second copy. However, it will create a hard link so that the same file can be used by both applications. If one of the applications gets uninstalled, Windows will move the file so that it doesn't get lost.

Deployment flexibility

MSIX is the successor of AppX, the original packaging format introduced in Windows 8 for Store applications. The technology has evolved over the years, and while in the beginning it was only supported by Microsoft Store, today, MSIX can be deployed with a wide range of consumer and enterprise technologies, including the following:

- Sideloading, either locally or from a website or network share
- Microsoft Endpoint Manager
- Microsoft Intune
- Windows Package Manager

Also, MSIX has recently become the leading technology for deploying applications in cloud environments. Thanks to a feature called **MSIX App Attach**, today, you can use the same MSIX package to deploy an application either in a local environment or on Azure Virtual Desktop,

that is, the Microsoft platform to create remote desktop environments, which makes it possible to access your workstation from anywhere. You can learn more about this feature at **https://docs.microsoft.com/en-us/azure/virtual-desktop/what-is-app-attach**.

Additionally, MSIX enables automatic updates not only in managed environments such as Microsoft Store but also in unmanaged ones like a website. We'll learn about this in more detail in the *Publishing your application to Microsoft Store* section.

Tamper protection and signature enforcement

MSIX packaged applications are installed inside a special folder, `C:\Program Files\WindowsApps`, which is a system-protected folder. Only Windows can modify or delete the content of this folder. Additionally, when an application is installed or updated, Windows Stores the hash of each file that is included inside the package. When the application starts, Windows double-checks that the hash of these files hasn't changed. If that happens, it means that a malicious actor has changed the content of the package; therefore, Windows will block the execution. The user must repair the application before using it again, which can be done in different ways based on the way it was deployed:

- If the application has been installed using Microsoft Store or a website, it will be automatically repaired by redownloading the package.
- If the application has been installed using an enterprise tool, it will be up to the IT Pro department to push the application to the affected devices again.

Additionally, certificates play a critical role to ensure the reliability of an MSIX package. A package must be signed with a certificate trusted

by the machine; otherwise, Windows will block the installation. Unlike other technologies (such as MSI), there is no way to install a package that isn't signed or that is signed with an untrusted certificate. We'll learn more about certificates in the *Publishing your application to Microsoft Store* section.

Registry virtualization

The Windows registry is one of the areas that is heavily affected by bad installation experiences. It's very common for applications to create new hives and keys in the registry, which are left in the system even when the application is uninstalled. This leads the registry to become fragmented, which tends to slow down Windows over time. To overcome this limitation, MSIX introduces the concept of registry virtualization, which is implemented in the following way:

- When the application creates or updates a registry key, the operation is performed against a virtual copy of the registry that only belongs to your application. If you open the **Registry Editor** screen, you won't see these keys.
- When the application reads a registry key, Windows will merge the system registry with the local virtual copy for your application. This approach enables your application to read both keys that you have created and keys that belong to the system configuration.

This feature helps to deliver a clean uninstallation experience. When the application is uninstalled, Windows will simply delete the virtual copy of the registry, avoiding leaving any orphan registry keys on the system.

However, this feature also introduces one of the potential limitations that you must consider when you plan to adopt MSIX. Since the virtualized registry is only visible to your application, this means that you can't create keys to share information with another application, since they won't be able to see them.

Local application data virtualization

One of the best practices for Windows development is to use the special **`AppData`** folder to Store data that is tightly coupled with the application. Configuration files, databases, and log files are just a few examples of data that only makes sense for your application, and, as such, they should be removed when the application is uninstalled. The **`AppData`** folder is the right place to Store these files since it lives inside the user-space; therefore, no matter whether the user is an administrator or has regular permissions, you can be safe in the knowledge that your application will always have the permission it requires to work with these files.

However, having the **`AppData`** folder in a different location from the installation leads to the same problems that we have seen with the registry. It's very common for an application to create some files in the **`AppData`** folder, just to leave them even after it has been uninstalled.

When an application is packaged with MSIX instead, all the reading and writing operations against the **`AppData`** folder are automatically redirected to the local storage of the application, which we learned about in ***Chapter 8***, *Integrating Your Application with the Windows Ecosystem*. The goal is the same as the registry virtualization: when the application is uninstalled, the local storage is automatically deleted, making sure that any unnecessary files aren't left behind, wasting space on the disk.

The Virtual File System (VFS)

Inside an MSIX package, you can create a special folder, called **`VFS`** (which stands for **Virtual File System**), that you can use to virtualize most of the system folders, such as **`C:\Windows`**, **`C:\Program Files\`**, and more. When your application runs and it looks for files in one of these folders (for example, it's very common when you have a dependency on a runtime or a framework), first, Windows will search for

them inside the VFS folder. If it doesn't find them, it will fall back to the actual system folders. Thanks to the VFS, you can solve two familiar challenges around application deployment:

- **Conflicts with existing dependencies**: Often, a problem encountered here is **DLL Hell**. Let's consider the following scenario: you have an application A, which requires framework XYZ to properly work. This framework is installed system-wide, and it's shared by all the applications that require it. At some point in time, you install application B, which requires a newer version of framework XYZ that overwrites the existing one. Application B will run fine, but the new version of the framework contains multiple breaking changes that will prevent application A from running properly. Thanks to VFS, you can solve this problem by including the framework inside the package. Windows will automatically use the embedded version instead of the system-wide one, preventing any conflict.
- **Creating self-contained packages**: Thanks to VFS, you can include all of the dependencies that you need inside the package, removing the requirement for your users to manually install runtimes and frameworks to install your application.

On the following page, **https://docs.microsoft.com/en-us/windows/msix/desktop/desktop-to-uwp-behind-the-scenes**, you can find a table with all of the system folders supported by VFS and the name you must use to map them inside the VFS folder.

Now that we have learned all the main features of MSIX, let's see the things we have to bear in mind before choosing to adopt MSIX.

MSIX limitations

At the time of writing, MSIX doesn't provide all the capabilities that are supported by traditional MSI installers. Some of them will likely never be supported since they are incompatible with the goal of MSIX

of providing reliable and safe installation technologies. Let's see the most important limitations to bear in mind and examine how we can overcome some of them:

- Since the registry and the application data folder are virtualized, you can't use them to share data across multiple applications. If you have this requirement, there are a few available options:
  - Starting from Windows 11, you can enable flexible virtualization. You can specify a list of folders under **`AppData`** and registry keys under the HKCU (Current User) hive that won't be virtualized but will be created in the system. You can learn more about this feature at **https://docs.microsoft.com/en-us/windows/msix/desktop/flexible-virtualization**.
  - You can disable virtualization completely by using a restricted capability called **`unvirtualizedResources`**. However, this capability will add some restrictions to the way you can deploy this package. You won't be able to publish it to Microsoft Store or via AppInstaller, but only using PowerShell.
  - If you are an IT pro, Windows 11 has introduced a feature called **Shared Package Container**, which you can use to share the same content across multiple applications. This way, they will be able to access the same registry keys and the same files, even if virtualized. You can learn more about this feature at **https://docs.microsoft.com/en-us/windows/msix/manage/shared-package-container**.
- You can't install software or hardware drivers. Drivers must be certified and published through Windows Update.
- You can't make changes to the Windows configuration. Since the application runs inside a virtualized environment, operations such as creating environment variables or enabling a Windows feature aren't supported.
- You can't run scripts or custom tasks. The MSIX deployment process can only copy the package content on the disk. If you need to run scripts, you can evaluate the adoption of the Package Support Framework, which is a library that you can integrate into your

package to overcome some of the MSIX limitations. One of the features supported by this library is to run scripts, with the following limitations:

- Scripts can be run only when the application starts or quits. They can't be executed as part of the deployment.
- Scripts will run inside the virtualized environment, so you still won't be able to perform tasks such as setting an environment variable.

You can learn more about this option at **https://docs.microsoft.com/en-us/windows/msix/psf/run-scripts-with-package-support-framework**.

- You can't deploy files outside of the installation folder. The content of an MSIX package is automatically copied inside a dedicated folder in **`C:\Program Files\WindowsApps`**. If your application needs to access files outside the installation folder, you can do the following:
  - Change the code of your application so that the files are copied at the first execution.
  - Use the Package Support Framework to run a script that copies the files you need outside of the installation folder.
  - Starting from Windows 11, you can use an extension called **`MutablePackageDirectories`**, which you can use to project a file into a system folder: **https://docs.microsoft.com/en-us/windows/msix/manage/create-directory**.
    - If it's a read-only operation, you can include the file in one of the special folders supported by the VFS.
- You can't write any file in the installation folder since it is system-protected. The best solution to overcome this limitation is to change your application's code to avoid using the installation folder to store files. If you can't make this change, you can use the Package Support Framework to redirect all the writing operations against the installation folders to the local storage of the application. This

solution is documented at **https://docs.microsoft.com/en-us/windows/msix/psf/psf-filesystem-writepermission**.

Now you have all the tools that you need to understand whether MSIX is the right technology for your application. It's time to learn more about how an MSIX package is structured.

The anatomy of an MSIX package

Compared to other deployment technologies, MSIX is a quite simple format. In fact, an MSIX package is just a compressed file, which you can open with any popular compression utility such as 7-zip or WinRAR. Inside an MSIX package, you will find the following:

- The files that make up your application, such as binaries, executables, DLLs, and assets.
- The manifest file, called **`AppxManifest.xml`**. We have met this file multiple times across the book. It describes the application, the extensions, and the features it uses. It's essential for Windows to manage the life cycle of the application. For example, through the manifest, Windows can detect whether it's the first installation or whether it's an upgrade.
- The blockmap file, called **`AppxBlockMap.xml`**. This contains the list of all the files included in the package along with their hashes. This file is used to light up some of the features we have described so far, such as tamper protection, differential updates, and disk space optimization.

The metadata inside the manifest file is used to determine two key information instances to identify an MSIX package:

- **Package Family Name (PFN)**: This identifies the lineage of the package. For example, the PFN of Microsoft Store, one of the built-in Windows apps, is **`Microsoft.WindowsStore_8wekyb3d8bbwe`**. It's composed of the publisher's name (**`Microsoft`**), the application

identifier (`WindowsStore`), and a hash, which is calculated based on the publisher. This makes it impossible for two different companies to create an application that has the same PFN. The folder where Windows creates the local storage of the application uses the PFN as the name.

- **Full Package Name**: This identifies a specific version of the package installed on the machine. Other than the information already described by the PFN, it also includes the version number and the CPU architecture. For example, at the time of writing, the full package name of Microsoft Store is `Microsoft.WindowsStore_22112.1402.3.0_x64__8wekyb3d8bbwe`. Windows uses the full package name for the name of the folder where it Stores the content of the package (inside `C:\Program Files\WindowsApps`).

The identity of an application can be customized by using the Visual Studio manifest editor, which is automatically opened when you double-click on the `Package.appxmanifest` file. You will find all the options under the **Packaging** section, as you can see in the following screenshot:

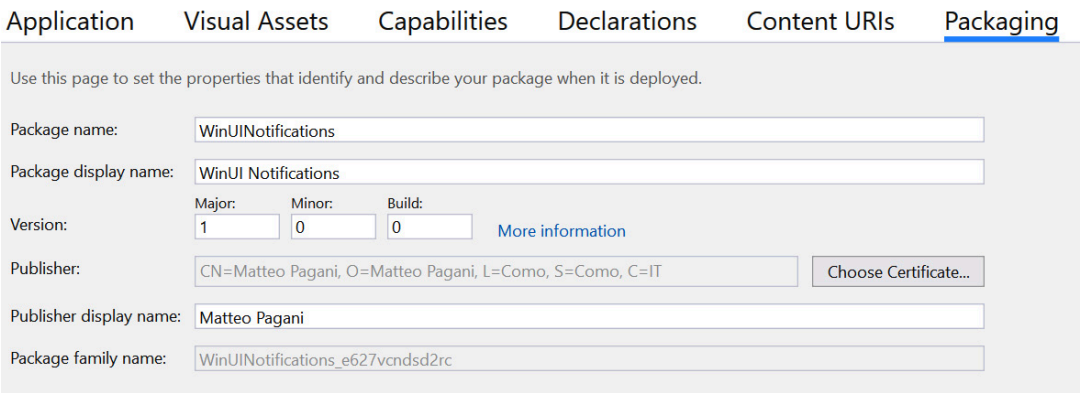

Figure 11.1 – The Packaging section of the manifest editor in Visual Studio

The fields that are used to calculate the PFN are **Package name** and **Publisher**. When you change them, you will observe that the **Package Family Name** field at the bottom will also change. Additionally, notice

how the **Publisher** field can't be freely edited, but there's a **Choose Certificate...** button on the right-hand side. As we're going to learn when we talk about signing, this is one of the MSIX requirements to satisfy: the **Publisher** field in the manifest must always match the subject of the certificate that you're going to use to sign the package.

Now that we have learned all the features of an MSIX package, it's time to see how to create one.

Creating and signing an MSIX package

There are three approaches that you can use to create an MSIX package:

- **Using Visual Studio**: This is the approach that we have seen in all the various chapters of the book so far. If you're either using the MSIX single-project or the Windows Application Packaging Project, you can quickly generate an MSIX package from your Windows application directly from Visual Studio. This is the approach we're going to explore in this chapter since it's the most suitable one for a developer.
- **Using the MSIX Packaging Tool**: This is a tool that has been released by Microsoft. It enables you to repackage any kind of application as MSIX. Using a special driver, the application captures any changes made to the system by a traditional installer and Stores them inside the MSIX package. It's mainly aimed at IT pros since you can use it to repackage applications for which you don't own the source code. You can learn more about this tool (including the link to download it) at **https://docs.microsoft.com/en-us/windows/msix/packaging-tool/tool-overview**.
- **Using a third-party tool**: In the last few years, most of the popular tools to create installers, such as Advanced Installer, InstallShield, or Wix, have added support for MSIX. By using the same project

you use today to generate a traditional installer with MSI, you can also create an MSIX package. You can find the list of all the partners at **https://docs.microsoft.com/en-us/windows/msix/partners**.

If you have followed the book so far, your WinUI, WPF, or Windows Forms application should already be set up for MSIX packaging. We explored the various options in ***Chapter 1***, *Getting Started with the Windows App SDK and WinUI*:

- In the case of a WinUI application, the MSIX generation is provided by the single-project MSIX, which is used when you create a new packaged application using the Visual Studio template.
- In the case of a WPF application or a Windows Forms application, you can package it with MSIX using the Windows Application Packaging Project.

Regardless of your choice, if you right-click on the project in Visual Studio, you will find an option called **Package and Publish** (if you're using the single-project MSIX template) or just **Publish** (if you're using the Windows Application Packaging Project). Inside this menu, you will find the **Create App Package** option, which will start a wizard that we can use to generate an MSIX package. Let's see the various steps in more detail.

Choosing a distribution method

The first choice in the wizard is how you want to distribute your application:

- **Microsoft Store**: This option generates a package optimized for Microsoft Store. This means the following:
  - The signing step will be skipped since Microsoft Store will automatically take care of the signing process as part of the package submission.

- In the next step, we'll be asked to log in with the Microsoft account associated with our developer account in Microsoft Store. Then, Visual Studio will show a list of the names we have reserved, along with the option to create a new one so that the generated identity can be automatically injected inside the manifest.

We'll learn more about Store publishing later in this chapter.

- **Sideloading**: This option is for all the other scenarios: distributions via a website, a network share, an enterprise tool, or even just mailing the MSIX package to a coworker.

Then, you can move on to choose the signing method.

Choosing a signing method

Thanks to this step, we can choose the certificate we want to use to sign our package. As mentioned earlier, this step will be skipped if we have chosen Microsoft Store as a distribution method. We'll be able to pick a certificate from multiple sources such as the following:

- Azure Key Vault: This is a cloud service where we can safely host protected information, such as secrets, passwords, and certificates. You can learn more at **https://azure.microsoft.com/en-us/services/key-vault/**.
- The local Windows certificate Store.
- A PFX file with the certificate.
- A new self-signed certificate.

You can observe these options in the following screenshot:

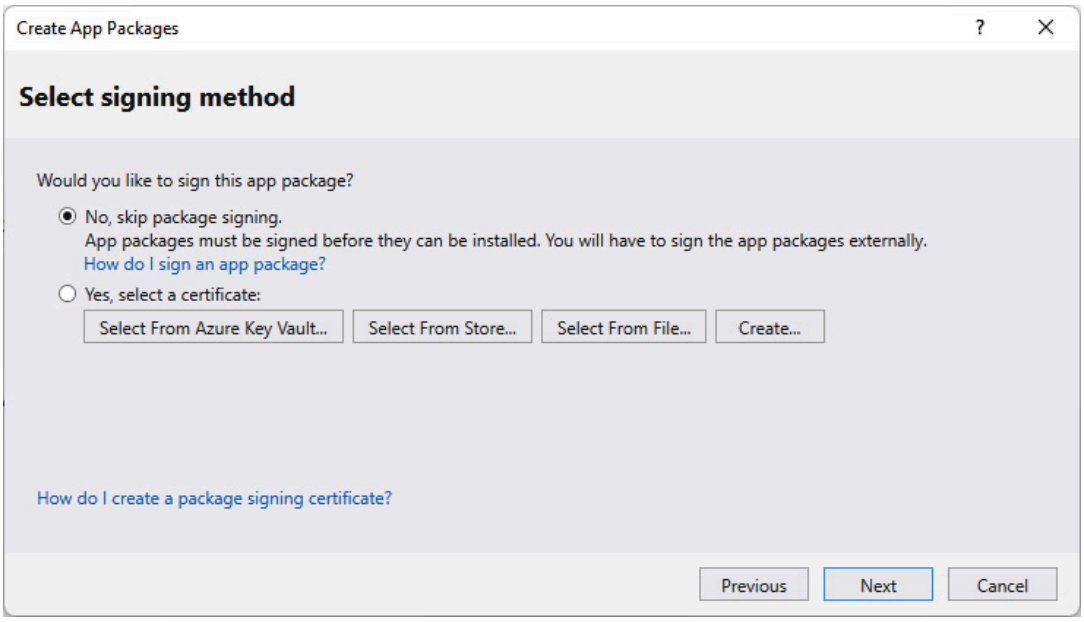

Figure 11.2 – The available signing options during the MSIX generation

We'll talk about the signing options, in more detail, later.

The next step will help us to define the content of the package.

Selecting and configuring packages

This section is very important as it defines the content of the package. This is what the section looks like:

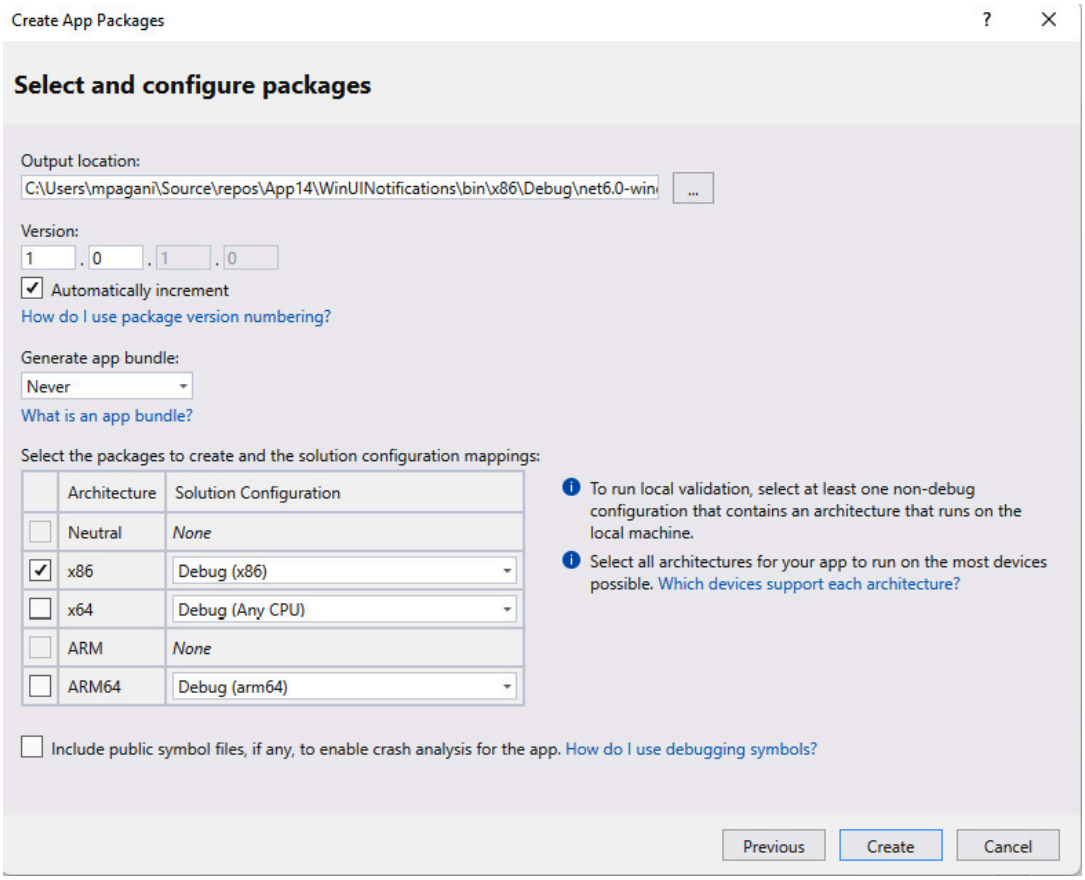

Figure 11.3 – The option to configure the packages you want to generate

From the preceding screenshot, we observe that we can first configure the following settings:

- **Version**: This is the version number associated with this release, which is stored in the manifest. The version must follow the **`X.Y.Z.0`** syntax: you can change the first three digits, but the last one must always be equal to **`0`**. It's important to always increase the version number when you generate a new package since it enables Windows to use it to update an existing installation. For this reason, you also have an option (**Automatically increment**) that will take care of this for you.
- **Generate app bundle**: In this book, we learned that the Windows App SDK is a native runtime. As such, applications that use it must be compiled for a specific CPU architecture. This means that if you want to build an optimized version for each CPU, you must create and distribute multiple packages. By generating an app bundle, you

can generate a single package that supports all CPU architectures. The package size will be bigger, but Windows will automatically download only the binaries targeting the CPU architecture of the current device.

The second section is needed to define what packages you want to create, based on the CPU architecture. The Windows App SDK supports x86, x64, and ARM64. For each of them, you can choose the configuration mode (either **Debug** or **Release**).

You have reached the end of the wizard: now you can hit **Create**, and Visual Studio will build your application and create the MSIX package. Once the operation has finished, a prompt will give you a link to directly access the folder where the package has been created:

Figure 11.4 – The wizard is complete: the MSIX package has been generated

If you double-click on the MSIX package, Windows will open the user interface (provided by the AppInstaller application, which we're going to see in more detail later) that you can use to install the application on the machine:

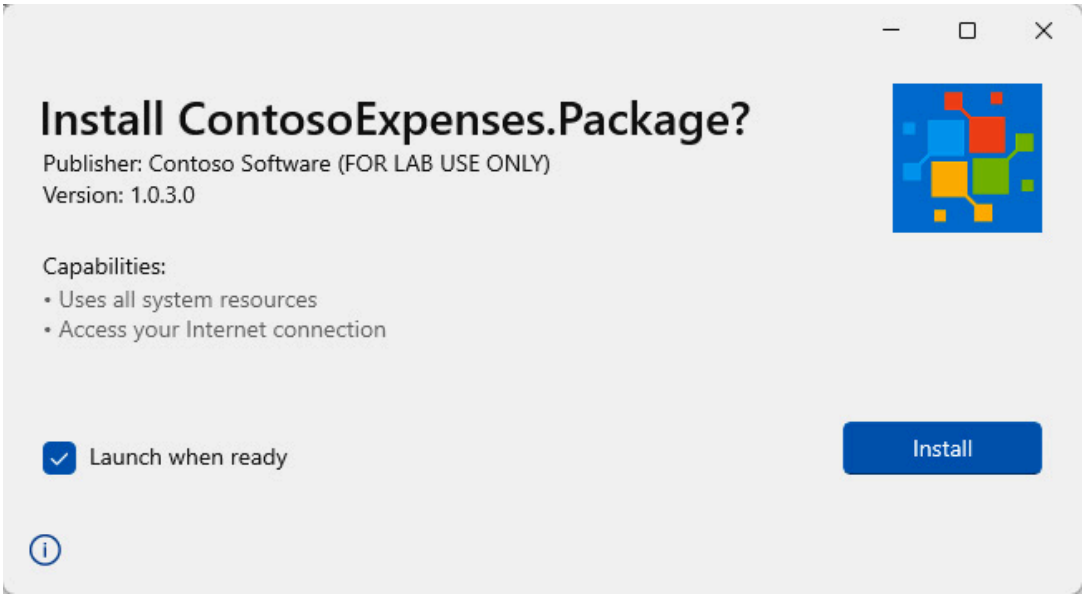

Figure 11.5 – The prompt to install an MSIX package

However, the application can be installed only if the package is signed with a trusted certificate. Let's learn more.

Signing an MSIX package

As already mentioned, signing is a key operation when you work with MSIX packages. If the package is not signed, users won't be able to install it.

This is the experience that users will get when they try to install a package that isn't signed or has been signed with a certificate that is not trusted by the current machine:

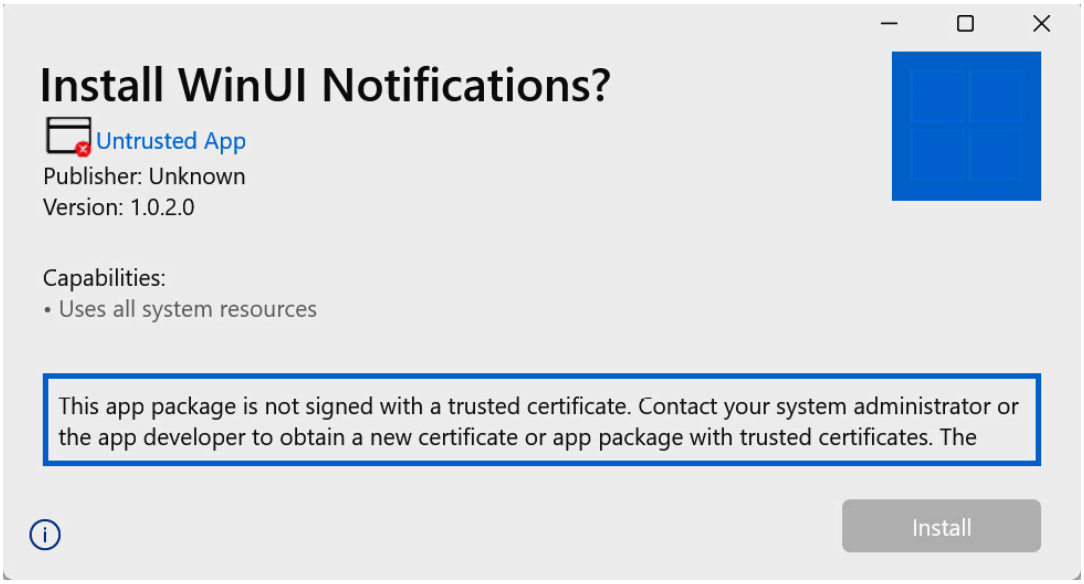

Figure 11.6 – The prompt to install an MSIX package that isn't signed or using a certificate that isn't trusted

There are three ways to obtain a code signing certificate:

- **A public certificate authority** (such as Digicert or Comodo): These certificates are the best fit for applications that must be publicly distributed since they are automatically considered to be trustworthy by Windows. When you buy one of these certificates, you must go through a strict process that verifies your identity. Thanks to this, the user won't have to take any special actions to install your MSIX package.
- **An enterprise certificate authority**: This is the most common scenario for enterprise distribution. These certificates are released to all the devices of the company, which makes them a good fit for signing applications that must be deployed internally.
- **With a self-signing certificate**: These certificates can be manually created on your local machine, without involving a certificate authority. However, they can only be used for testing scenarios since, by default, they aren't considered to be trustworthy by Windows. Unlike the ones released by a public certificate authority, you can create them with any random name, since the identity isn't validated. Users must manually add them to the **Trusted People** category in the certificate store, which requires administrative rights.

Microsoft is also working on a new service, called **Azure Code Signing Service**, which will make the signing experience easier and cheaper. One of the downsides of public certificates is that they can be quite expensive, and they must be renewed periodically. The service isn't publicly available yet, but if you want to learn more, you can watch the following video from the MSIX team at **https://www.youtube.com/watch?v=Wi-4WdpKm5E**.

The key requirement to satisfy when you use a code signing certificate to sign a MSIX package is that the subject of the certificate must match the publisher entry in the manifest. Visual Studio will automatically set the correct value when you import a certificate. For example, if you explore any application published by Microsoft, you will find that the publisher in the manifest is set with the following value:

```
CN=Microsoft Corporation, O=Microsoft Corporation,
  L=Redmond, S=Washington, C=US
```

A vital part of the signing process is the timestamp. Certificates have an expiration date, which can lead to a package to stop working after it's been passed. By specifying a timestamp server, you'll be able to continue installing and using the package even after the certificate has expired. You can find a list of available time stamp servers at **https://gist.github.com/Manouchehri/fd754e402d98430243455713efada710**.

If you're a developer working with Visual Studio, the easiest way to sign an MSIX package is to choose your certificate in the manifest editor or in the wizard we have previously seen to create an MSIX package, which supports all the options we have seen so far, including setting a timestamp server. The manifest editor will also give you the option to generate a self-signing certificate on the fly, if you don't have one, by clicking on the **Create...** button:

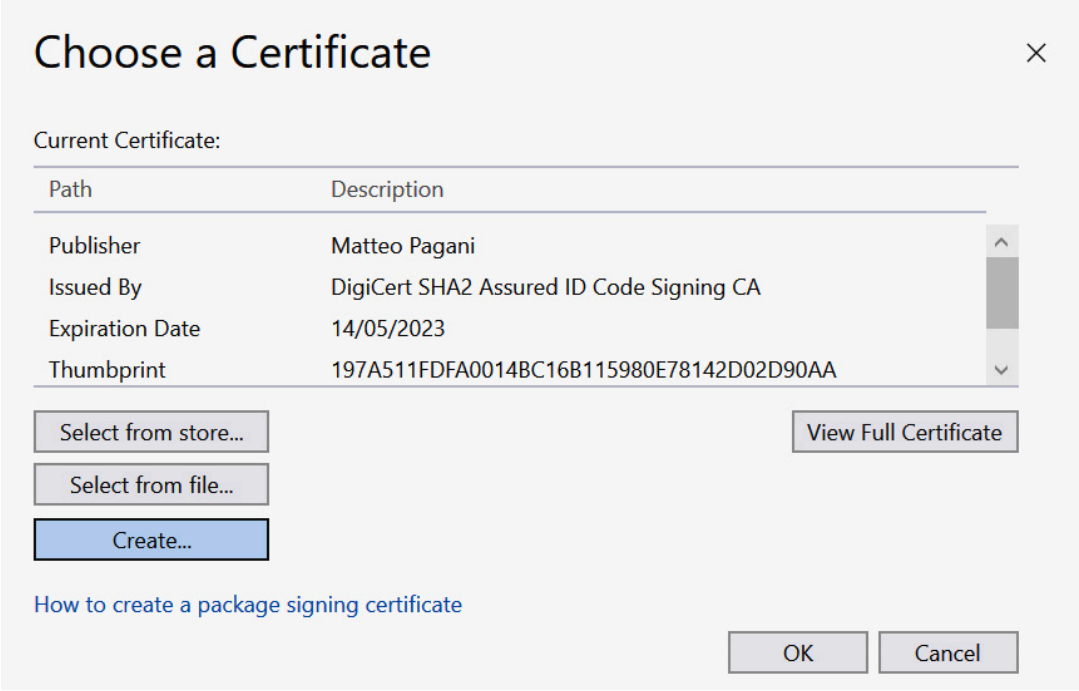

Figure 11.7 – Choosing a certificate

After you have chosen the certificate, Visual Studio will automatically sign the package as part of the build process when you create an MSIX package for distribution using the Create App Packages wizard.

Other options to sign an MSIX package include the following:

- **Using the signtool utility**: This is included in the Windows SDK. It's a command-line utility whose usage is described at **https://docs.microsoft.com/en-us/windows/msix/package/sign-app-package-using-signtool**.
- **Using a third-party tool such as MSIX Hero**: This is an application developed by Marcin Otorowski, which gives you multiple tools to work with MSIX, such as browsing the catalog of packages deployed on your machines, unpacking an MSIX package, and more. Among the available options, you have also the opportunity to sign an MSIX package given a valid certificate. You can download this from **https://msixhero.net/**.

Signing the package with Visual Studio is a good solution when you're in the development and testing phases. However, it isn't suitable for more professional scenarios in which you want to automate the build-

ing and deployment of your application. This is because it forces you to share the certificate as part of the project, which might lead to identity theft.

In the next chapter, we'll explore other ways to sign your package. Now, it's time to talk about the distribution options.

Publishing your application to Microsoft Store

Microsoft Store is the most efficient and straightforward way with which to distribute your application to a broader audience. It's a catalog of applications and games that is preinstalled on every Windows computer, which greatly simplifies the experience of downloading and installing applications on your machine. Users no longer have to open a browser, search for an application with a search engine, find the right website (avoiding, in the meantime, potentially malicious ones), download the installer, and execute it. With Microsoft Store, they can simply search for the app they need and, by clicking on a button, trigger the installation. The following screenshot shows Microsoft Store in the browser:

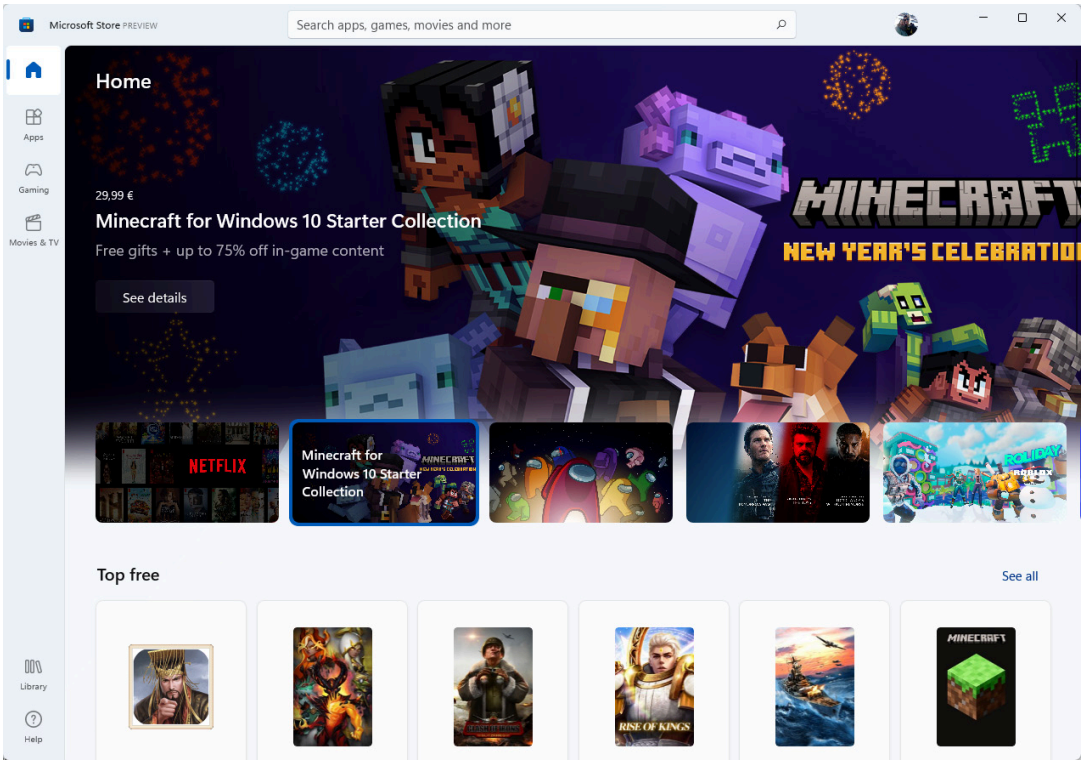

Figure 11.8 – The Microsoft Store main page

As a developer, Microsoft Store greatly simplifies the distribution experience thanks to the following features:

- You don't need to buy a certificate and manually handle the signing. The Store will automatically sign all the applications with a trusted certificate issued by Microsoft.
- By default, the Store offers automatic updates. You don't have to implement a custom solution, but it's enough to publish an update on the Store to automatically distribute it to your customers.
- You can leverage a built-in monetization infrastructure. You can sell your application or add-ons through Microsoft Store, rather than having to build your own licensing infrastructure. You just need to set your price during the submission process; users will pay the application using the payment systems that are connected to their Microsoft account, such as a credit card. Microsoft will keep a fee of 15% for every purchase. If you want to keep 100% of the income, or if you have your own licensing system, you aren't obligated to use the one provided by Microsoft.

- You have access to advanced deployment features, such as gradual rollout and package flights to deploy different versions of the same app to different users (such as alpha, beta, and more).
- You have built-in support for marketing activities, such as a dedicated window for your application with descriptions, screenshots, and videos; the ability to generate promotional codes to give away an application for free; and support for a sales campaign in which the application price is automatically reduced during a given period of time.
- You can engage with your customers by collecting feedback, reviews, and ratings.

The starting point to publish your Windows applications to Microsoft Store is by creating a developer account at **https://developer.microsoft.com/en-us/microsoft-store/register/**.

There are two types of accounts, as follows:

- **Personal**: These accounts are linked to your personal identity. To open it, you are asked to pay a one-time fee of $19. At the end of the process, the account will be ready to use immediately.
- **Business**: These accounts are linked to a business entity, such as a company. In this case, the one-time fee is $99. This type of account takes a bit longer to be opened since a vetting company will verify your identity and will make sure that the company exists and that you can operate on its behalf.

Regardless of the type you choose, a developer account must be linked to a personal Microsoft account. You can't open it using a work account, such as the one provided by Microsoft 365. However, once the developer account has been created, you can link it to an Azure Active Directory tenant (such as the one that belongs to your company). This is so that you can enable coworkers to access the developer portal. Additionally, you can assign them distinct roles: for example, a developer can only submit apps, whereas a finance contributor can see all

the financial reports. You can learn more about this configuration at **https://docs.microsoft.com/en-us/windows/uwp/publish/manage-account-users**.

Once your account has been set up, you can log in to the **Partner Center** portal at **https://partner.microsoft.com/dashboard** and reserve a new name, which must be unique. The name will also be used to generate the PFN, which defines the application's identity. Once the name has been reserved, you can create a new submission, where you can provide all the information that will be made visible to users: pricing, distribution markets, description, age rating, screenshots, and more. You can find a detailed description of the submission flow at **https://docs.microsoft.com/en-us/windows/uwp/publish/**.

However, there's one step that is significantly different based on the deployment technology you choose. In fact, Microsoft Store supports submitting both packaged applications and unpackaged applications. This is a new feature that has been introduced by the new Microsoft Store version launched in Windows 11, which is now also available on Windows 10.

Let's see the specific details about how to manage both scenarios.

Submitting an MSIX packaged application

The MSIX packaged model is the most powerful one since it gives you access to all the features offered by Microsoft Store. In fact, an MSIX packaged application is hosted directly by the Microsoft infrastructure, which enables all the features highlighted at the beginning of this section regarding deployment, such as a built-in signing process, automatic updates, and gradual rollouts.

To generate an MSIX package for the Store, you can use the Visual Studio wizard that you can trigger by right-clicking on your project and choosing **Publish** | **Create app packages**. In the first step, choose

Microsoft Store under a new app name. You will be asked to log in with your developer account so that Visual Studio can list all the names you have reserved. Choose the one you have created for your application and continue the wizard, following the steps we described earlier in this chapter.

The outcome will be an unsigned MSIX package (remember that the Store will take care of it for you) and the identity assigned by Partner Center. The package will have a special extension—**`.msixupload`**. Now you can navigate to the **Packages** section of the submission process and upload it. The Store will ingest it and make it available through its platform.

Now, let's see how you can handle unpackaged applications.

Submitting an unpackaged application

In the case of an unpackaged application, the Store will function mainly as a window for your software. Users will still be able to find your application in the catalog, discover information about it, and install it with a single click. However, the application won't be hosted by the Store. Instead, it must be hosted by your own infrastructure, such as a website or cloud storage. Since the Store doesn't handle the distribution, many of the features won't be available since it behaves like a sideloading scenario: you must take care of signing or delivering automatic updates. However, you will still be able to use the monetization services and the feedback system provided by the Store.

NOTE

At the time of writing, publishing unpackaged applications is in preview. Before getting access to this feature, you must submit your nomination at **https://aka.ms/storepreviewwaitlist**.

When you choose to publish an unpackaged application, the **Packages** section of the submission process will be different. Instead of uploading a package, you must provide the following information:

- The package URL: This is the URL that points to the MSI or EXE installer of your application. Ideally, you should use a CDN to make sure that you can manage the traffic that will be generated by the Store.
- The languages that your application supports: You will have the option to provide different metadata (such as description, screenshots, and more) for each language.
- The CPU architecture is supported by your installer.
- The Store must be able to install your application in a silent way, without user intervention. This is typically achieved by passing a parameter such as `/s`, `/q`, or `/quiet`. As part of the submission, you must specify this parameter.

Regardless of the deployment technology you choose, the application will go through a certification process.

The certification process

Once you submit the application, it won't be immediately available, but it must pass a certification process. The Store will run a series of automated tests that will make sure your application behaves properly: for instance, it doesn't crash at startup, it doesn't contain malicious code, it's reliable, and more. Additionally, your application might also be selected for manual testing, which will validate not only the technical side but the content policies too. In fact, some content isn't allowed in the Store, such as pornography, blasphemy, and the like.

You can review all the policies at **https://docs.microsoft.com/en-us/windows/uwp/publish/store-policies**.

Once your app has been certified, it's made available to all Windows users. At this point, you will also have the option to submit updates.

Submitting updates

To submit an update, you must create a new submission starting from an existing application. The submission process is the same as the one you have followed first the first release, with the difference that all the various steps (for instance, pricing, markets, metadata, and more) will be already filled with the information you have submitted the first time.

The way you submit an updated package changes based on the deployment scenario:

- For MSIX packaged apps, you have built-in support for automatic updates. You just need to create a new package from Visual Studio, set a higher version number, and upload it to the **Packages** section of the submission process.
- For unpackaged apps, the Store isn't able to provide automatic updates. You can only use the **Packages** section if you need to change the package URL. You must provide a built-in update feature in your application if you want to automatically deliver updates to your customers.

Microsoft Store is a powerful distribution system for consumer applications. However, for enterprise applications, it might not be the best option, since many of the features it provides are tied to the consumer ecosystem. For instance, the payment system is tied to the personal Microsoft account and enterprises are unable to build their own catalog of applications.

For all the other scenarios, you can opt for sideloading distribution from a website, cloud storage, or an internal share.

In the next section, we're going to learn how, thanks to MSIX and AppInstaller, we can retain some of the features that we have seen so far.

Deploying your applications from a website using AppInstaller

If you are building a Windows application, there are many reasons why you might want to distribute your application using a website. If you are an **Independent Software Vendor** (**ISV**), you might only need to protect the download of the application to your registered users by using a reserved area on your website; if you are an enterprise, you might build an internal website or an internal network share where employees can download the applications they need.

Of course, if you prefer to use the unpackaged model, you can keep using traditional deployment technologies such as MSI or ClickOnce and add a link on your website to download it. However, thanks to MSIX, you can enable a more streamlined deployment experience using AppInstaller. This enables you to do the following:

- Deploy dependencies and optional packages automatically.
- Deliver automatic updates through your website or network share.
- Support automatic remediation if the application is corrupted.

AppInstaller is an application included in Windows 10 that enables all these scenarios other than providing the installation experience for MSIX packages. The user experience you see when you double-click on an MSIX package to start the installation is managed by AppInstaller.

Additionally, AppInstaller supports a special file, with the `.appinstaller` extension, which describes the installation experience of your MSIX package. You can specify the URL where the package can be downloaded, where to download eventual dependencies, such as

the Windows App SDK runtime, and how you want to handle updates. This file can be published together with your MSIX package or embedded directly inside it.

Let's see what an AppInstaller file looks like:

```
<AppInstaller
Version="1.0"
Uri="https://www.contoso.com/ContosoExpenses
  .appinstaller "
     xmlns="http://schemas.microsoft.com/appx/appinstal
ler
        /2018">
  <MainPackage
    Name="ContosoExpenses"
    Publisher="CN=Matteo Pagani, O=Matteo Pagani,
L=Como,
      S=Como, C=IT"
    Version="1.0.5.0"
    ProcessorArchitecture="x86"
    Uri="https:\\www.contoso.com\ContosoExpenses.msix"
/>
  <Dependencies>
    <Package
      Version="0.319.455.0"
      Uri="https://www.contoso.com/Microsoft.WindowsApp
        Runtime.1.0.msix"
      Publisher="CN=Microsoft Corporation, O=Microsoft
        Corporation, L=Redmond, S=Washington, C=US"
      ProcessorArchitecture="x86"
      Name="Microsoft.WindowsAppRuntime.1.0" />
  </Dependencies>
  <UpdateSettings>
    <OnLaunch
      HoursBetweenUpdateChecks="0"
```

```
        ShowPrompt="true" />
    </UpdateSettings>
  </AppInstaller>
```

Let's break it down into the main four sections:

- The first key element is the **`Uri`** attribute of the main **`AppInstaller`** entry, which is the root of the XML file. This **`Uri`** attribute must point to the website or the network share where you are going to publish this AppInstaller file.
- The **`MainPackage`** entry describes the main MSIX package with its name, publisher, version number, and CPU architecture. Also, in this scenario, a key element is the **`Uri`** attribute, which must point to the location where you're going to publish your MSIX package. If you are distributing an app bundle instead of a single package, you can replace **`MainPackage`** with **`MainBundle`**.
- The **`Dependencies`** entry is used when an application has a dependency from a runtime or a library distributed with MSIX. The Windows App SDK runtime is a good example of such a dependency. Also, in this case, the key property is the **`Uri`** attribute, which points to the location of the dependency. Thanks to this section, we can install dependencies together with the application from a custom location, also supporting scenarios where Microsoft Store is blocked, so the dependency can't be downloaded automatically.
- The **`UpdateSettings`** section can be used to control the update logic. In this example, we are configuring our application to check for updates every time the application starts and, if an update is available, to show a notification prompt. Other available options are checking for updates in the background or making updates mandatory (which blocks the ability to run the application until the application has been updated).

Additionally, the AppInstaller file can specify other types of dependencies, such as optional packages (special packages that contain add-ons for the main application) or modification packages (special packages that can customize the existing installation of the application).

The following documentation gives you a detailed overview of the AppInstaller schema and all of the available options: **https://docs.microsoft.com/en-us/windows/msix/app-installer/how-to-create-appinstaller-file**.

However, there are better ways than having to generate the file manually. Let's explore the options we have next.

Generating an AppInstaller file with Visual Studio

Visual Studio can generate an AppInstaller file for you as part of the build process that generates an MSIX package. The starting point is the same wizard we looked at earlier, that is, the one that you can trigger by choosing **Publish** | **Create a package** when you right-click on your project.

NOTE

At the time of writing, this option is only available for the Windows Application Packaging Project. If you're building a WinUI application using the single-project MSIX approach, you will still see the option to generate an AppInstaller file, but it won't have any effect. At the end of the build process, the AppInstaller file will be missing.

When you choose **Sideloading** as a distribution method, you can also enable a flag called **Enable automatic updates**, as you can see in the following screenshot:

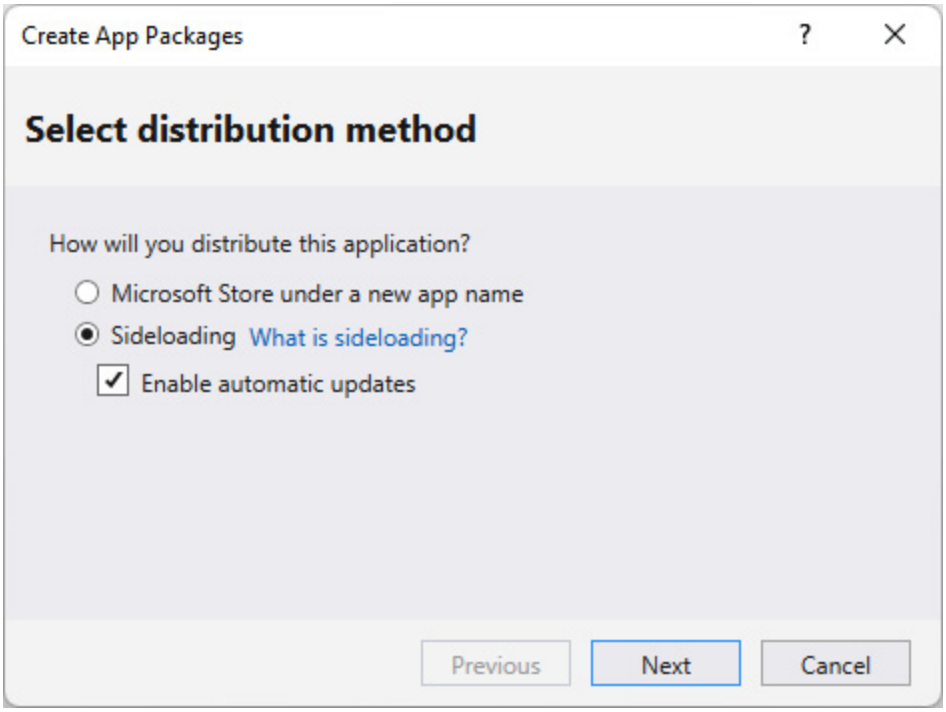

Figure 11.9 – The option to enable the creation of the AppInstaller file during the MSIX generation

When you check this option, you will see a new step of the wizard, titled **Configure update settings**, at the end of the process. It will appear before the creation of the package:

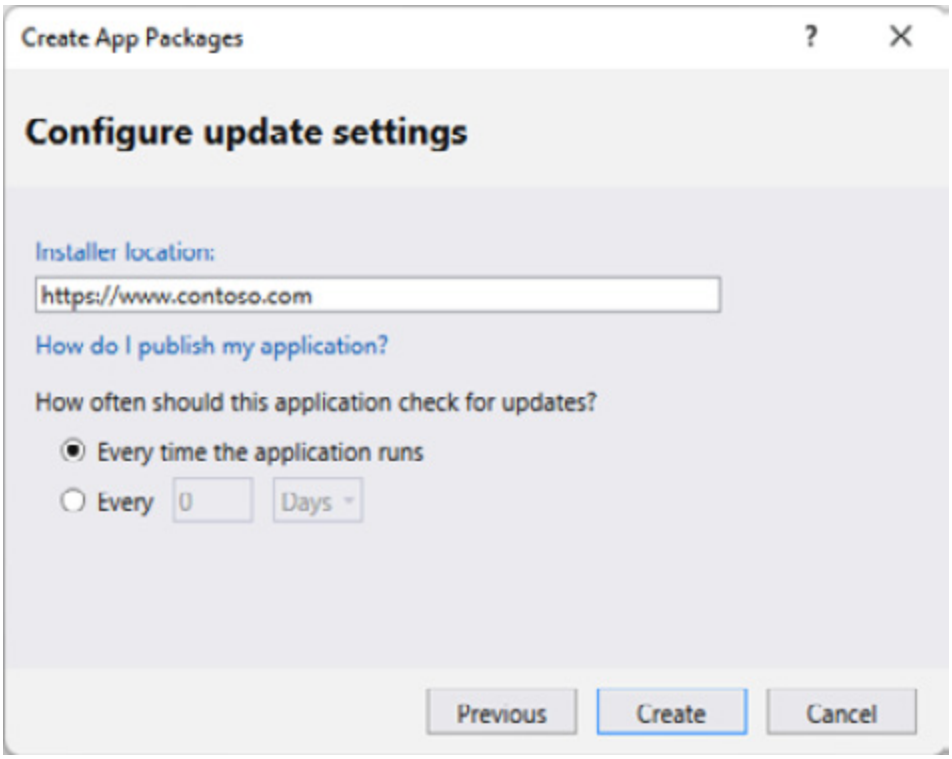

Figure 11.10 – The step in the wizard to configure the update settings

In this step, you must provide the URL of the network share path where you're planning to make your application available and how often Windows should check for updates. Then, you can hit **Create** to start the creation of the MSIX package.

At the end of the process, in the same folder where Visual Studio has generated the MSIX package, you will also find the following two additional files:

- One with the **`.appinstaller`** extension, already configured in the proper way by combining the information you have provided in the wizard (such as the installer location) and the ones generated by Visual Studio (such as the name of the MSIX package).
- Another is called **`index.html`**. This is a static web page that you can use to trigger the installation of the application. Other than displaying information such as the logo, name, and version number, it includes a button that uses the **`ms-appinstaller`** protocol (we'll learn more about this later) to start the installation of the application.

However, this wizard has a limitation: the AppInstaller file has evolved over time by supporting new features such as update prompts or mandatory updates. The wizard didn't follow the same evolution and, as you have seen in *Figure 11.9*, it doesn't support these new features.

To work around this limitation, you can add an AppInstaller template to your project. This is a special file that will be used as a template during the generation of the package, and it contains placeholders for the key information, such as the installer location or the application's version. During the build process, Visual Studio will replace them with the actual data generated during the build. To add this file, you must right-click on your project and choose **Add** | **New item**. Choose the AppInstaller template, leave the default name (**`Package.appinstaller`**), and press **Add**. This is what the file looks like:

```
<AppInstaller Uri="{AppInstallerUri}"
    Version="{Version}"
    xmlns="http://schemas.microsoft.com/appx/
      appinstaller/2017/2">
  <MainPackage Name="{Name}"
               Version="{Version}"
               Publisher="{Publisher}"
               Uri="{MainPackageUri}"/>
  <UpdateSettings>
    <OnLaunch HoursBetweenUpdateChecks="0"/>
  </UpdateSettings>
</AppInstaller>
```

This is very similar to the file we have seen before, except that most of the information is described with a placeholder and wrapped inside curly braces (such as **`{AppInstallerUri}`** or **`{Publisher}`**). Being a template, you can customize it as you prefer, for example, by adding new entries and attributes in the **`UpdateSettings`** section to enable new features.

NOTE

Visual Studio generates the AppInstaller template using an old **`xmlns`** *schema, which doesn't support many of the new features. As such, it's important to replace the current one with the following:*

`xmlns="`[**http://schemas.microsoft.com/appx/appinstaller/2021**](http://schemas.microsoft.com/appx/appinstaller/2021)**`"`**

When you generate a package using the wizard, Visual Studio will use this file as a base template. In fact, in the **Configure update setting** section of the wizard, you won't be able to customize the settings (other than the location URL) anymore since they will be taken directly from the template.

But what if you want more flexibility when creating an AppInstaller file? Let's explore another option next.

Generating an AppInstaller file with AppInstaller File Builder

AppInstaller File Builder is an application developed by Microsoft, which is part of the MSIX Toolkit—a series of utilities to support working with MSIX packages. You can download it from https://github.com/microsoft/MSIX-Toolkit/releases/.

You can use this tool to generate an AppInstaller file more easily, thanks to its visual interface:

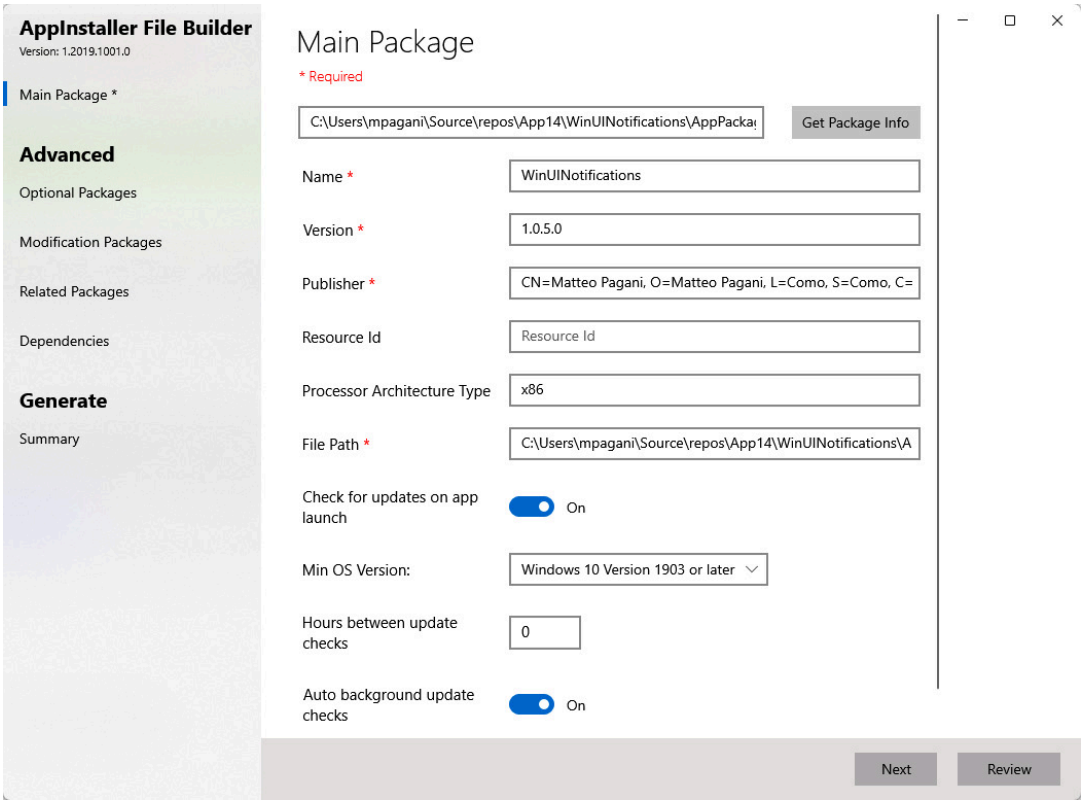

Figure 11.11 – The AppInstaller File Builder utility

What makes this a powerful tool is that instead of manually filling in all the required information, you can just click on the **Get Package Info** button to choose the MSIX package you want to publish from your hard disk. The tool will automatically extract all the information for you. Thanks to this tool, you can define the following:

- The main package

- The dependent packages (such as the optional packages, modification packages, related packages, and dependencies)
- The update conditions, which are based on the Windows version that you're targeting

Once you have filled in all of the information, you can move on to the **Summary** section. Here, you'll be asked to specify the URL where you're going to publish the AppInstaller file, the URL of the various packages, and the version number:

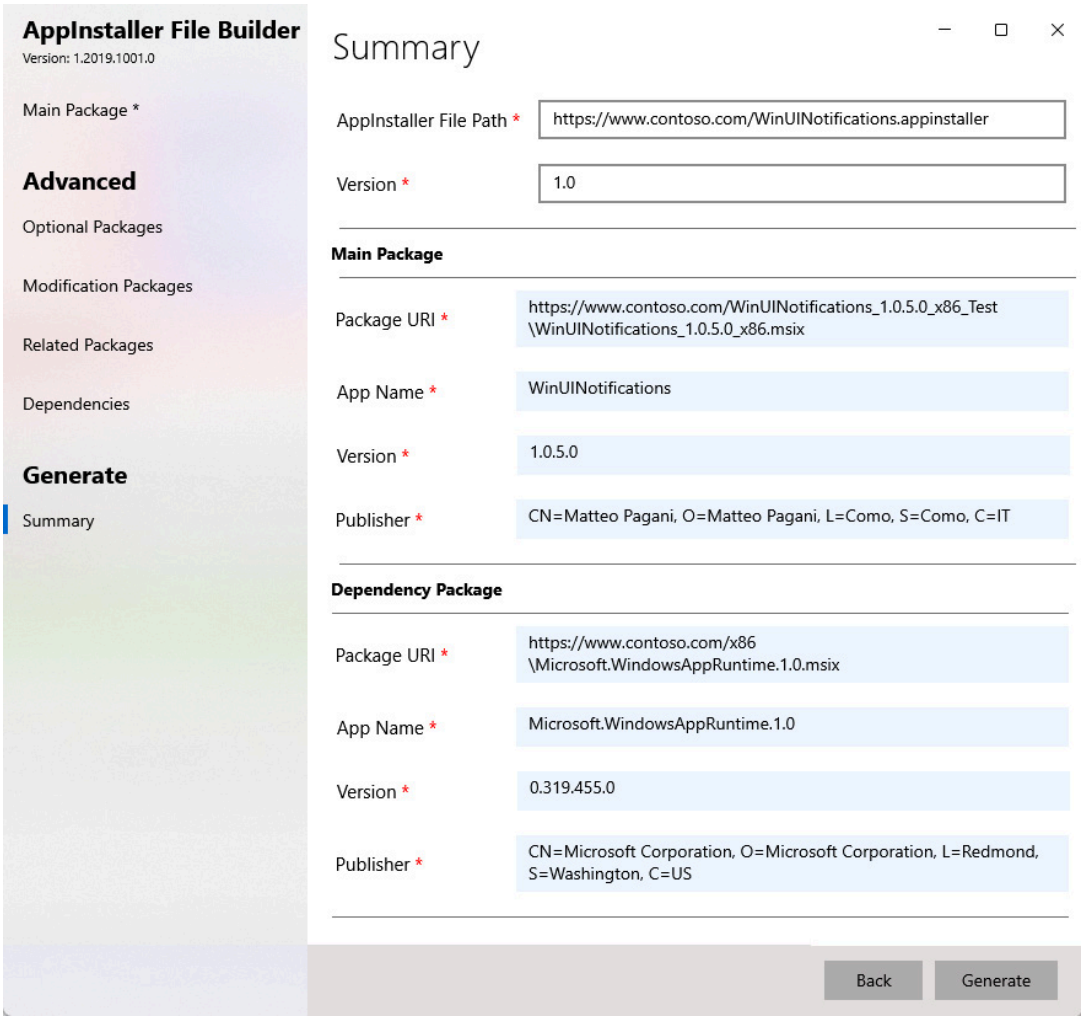

Figure 11.12 – The summary page to generate the AppInstaller file

By clicking on **Generate**, the tool will generate the AppInstaller file for you, all ready to be used. But how can we use it?

Deploying an AppInstaller file

Now that we have an AppInstaller file, all we need to do is deploy it together with our packages (the main one and the eventual dependencies) in the location we have chosen, which can either be an HTTP endpoint or a network share.

Once we have deployed them, we can use one of the following two ways to trigger the installation of the package:

- Add a link to our web page to download the AppInstaller file locally. The user must double-click on it to start the installation process. Windows will automatically download the MSIX package and its dependencies from the URLs that are specified inside the file.
- Using the **`ms-appinstaller`** protocol, which is the default approach used by Visual Studio. In this scenario, the link you add to your web page won't download the AppInstaller file locally, but it will point to it using the **`ms-appinstaller`** protocol (for example, **`ms-appinstaller:?source=`**https://www.contoso.com/ContosoExpenses.appinstaller). This approach provides the best user experience because the installation will be directly triggered from the website, saving the extra step of downloading the file locally and launching it.
  NOTE
  At the time of writing, Microsoft has discovered a vulnerability related to the **`ms-appinstaller`** *protocol, which is described at* **https://msrc.microsoft.com/update-guide/vulnerability/CVE-2021-43890**. *For this reason, option 2 isn't currently available, but it might be restored in the future once the vulnerability has been addressed.*

Starting from Windows 11, you can also choose to embed the AppInstaller file inside your MSIX package. This doesn't replace the need to publish your AppInstaller file to a website or network share: if you want to deliver automatic updates, you will still need to make it available through an HTTP endpoint. However, by embedding the

AppInstaller file inside the package, you can enable automatic updates even if the application has been manually installed the first time; you can do this by directly using the MSIX package rather than the AppInstaller file.

If you want to use this possibility, first, you'll have to copy the same AppInstaller file you previously generated and deployed inside the Visual Studio project. Then, you must declare it in the application's manifest.

First, you'll need to add an extra namespace called **`uap13`**:

```
<Package
    xmlns:uap13="http://schemas.microsoft.com/appx
      /manifest
        /uap/windows10/13" IgnorableNamespaces="uap13">
...
<Package>
```

Then, inside the **`Properties`** section of the manifest, you must add an **`AppInstaller`** entry with the path of the AppInstaller file inside the package:

```
<Properties>
    <uap13:AutoUpdate>
        <uap13:AppInstaller File="Update.appinstaller"
/>
    </uap13:AutoUpdate>
</Properties>
```

Now that we have deployed and installed our application via an AppInstaller file, let's see how we can apply updates.

Updating an application automatically

Thanks to the **`UpdateSettings`** section, we have added to our AppInstaller file. Now, we can enable automatic updates without hav-

ing to write any other code in our application. We just need to generate an updated AppInstaller file and MSIX package, making sure that we set a higher version number for both. Then, we need to upload them in the same HTTP or network location where we deployed the original version of the application.

Now, based on the update logic that we have defined in the AppInstaller file, Windows will automatically pick the update and install it. Users won't need to come back to the website or the network share to download and install the package again.

For example, if you are using the **`OnLaunch`** configuration and you have enabled the **`ShowPrompt`** option, users will see the following message the next time they open your application:

Figure 11.13 – The prompt that is displayed to the user when an update is available

In other scenarios, you might want more granular control over the updates. Let's explore this scenario by using the Package APIs.

Updating an application from code

After you have deployed an application using AppInstaller, you can leverage which APIs belong to the **`Windows.Management.Deployment`** namespace to check for available updates and install them. This approach requires more work since you'll have to write some additional code, but it enables you to further customize the update logic since you are in full control of the update process.

Let's see a code example that we can use to trigger the update:

```
private async Task CheckUpdatesAsync()
{
    PackageManager pm = new PackageManager();
    Package package =
pm.FindPackageForUser(string.Empty,
      Package.Current.Id.FullName);
    PackageUpdateAvailabilityResult result = await
      package.CheckUpdateAvailabilityAsync();
    switch (result.Availability)
    {
        case PackageUpdateAvailability.Available:
        case PackageUpdateAvailability.Required:
            var installTask =
              pm.AddPackageByAppInstallerFileAsync(
            new Uri("https://www.contoso.com/
              ContosoExpenses.appinstaller"),
                AddPackageByAppInstallerOptions
                  .ForceTargetAppShutdown,
pm.GetDefault
                    PackageVolume());
            installTask.Progress += (installResult,
```

```
                progress) =>
            {
                Console.WriteLine(progress);
            };
            var installResult = await installTask;
            if (!installResult.IsRegistered)
            {
                Console.WriteLine(installResult.ErrorTe
xt);
            }

            break;
        case PackageUpdateAvailability.Unknown:
        case PackageUpdateAvailability.NoUpdates:
            MessageBox.Show("No updates available");
            break;
    }
}
```

Here, we use the **`PackageManager`** class to retrieve a reference to the current instance of the application by calling the **`FindPackageForUser()`** method. As parameters, we must pass the identifier of the user (we can pass an empty string if we want to retrieve the packages for all of the users) and the Package Family Name. Since our application is packaged, we can retrieve it using the **`Package.Current.Id.FullName`** property.

Once we have a reference to the package, we can invoke the **`CheckUpdateAvailabilityAsync()`** method, which will use the information stored inside the AppInstaller file that we used to install the application, to check whether there's a new update available. If there's indeed a new version, we will get back the **`Available`** value or the **`Required`** value of the **`PackageUpdateAvailability`** enumerator (this depends on whether we marked the update as optional or required). If that's the case, we can move on with the installation process by calling

the **`AddPackageByAppInstallerFileAsync()`** method of the **`PackageManager`** class. This requires the following parameters:

- The URL of the AppInstaller file.
- The behavior to apply when the update is installed. In this example, by using the **`ForceTargetAppShutdown`** option, we force the application to be closed so that the update can be applied immediately.
- The volume where the application has been installed can be retrieved by calling the **`GetDefaultPackageVolume()`** method of the **`PackageManager`** class.

The result of the method is represented by a special Windows Runtime interface called **`IAsyncOperationWithProgress`**, which can be used to track the progress of asynchronous operations. In the previous snippet, you can see an example of how you can use it: we don't immediately execute the method, but we just Store a reference to the **`Task`** object that represents it (notice how we call the **`AddPackageByAppInstallerFileAsync()`** method without prefixing it with the **`await`** keyword). Then, we subscribe to the **`Progress`** event, which we can use to monitor the progress of the operation.

Following this, we start the task by invoking it with the **`await`** prefix. We can use the **`IsRegistered`** property of the result to determine whether the operation has been completed successfully; otherwise, we can use the **`ErrorText`** and **`ErrorCode`** properties to understand what went wrong.

So, what if you have opted to not use the AppInstaller file, but you still want to provide a way to deliver updates? In this case, you can use the **`AddPackageAsync()`** method offered by the **`PackageManager`** class, directly passing the URL of the updated MSIX package as a parameter. This is shown in the following snippet:

```
  PackageManager packagemanager = new PackageManager();
```

```
await packagemanager.AddPackageAsync(new
  Uri("https://www.contoso.com/ContosoExpenses.msix")
    ,null, AddPackageOptions.ForceApplicationShutdown);
```

Even if this code might look simpler than the AppInstaller scenario, it actually requires more work from your side. In fact, if the application hasn't been installed with an AppInstaller file, Windows doesn't have a way to understand whether there's a new package available. As such, you must implement this logic on your own, for example, by exposing this information through a REST API that you must call before installing the update.

Let's wrap up the AppInstaller overview by taking a quick look at two new features that have been added to Windows 11.

Supporting updates and repairs

Starting from Windows 11, AppInstaller supports two new features:

- The ability to specify multiple update URLs: This way, if one of them should go down, Windows will automatically fall back to one of the others.
- The ability to specify one or more repair URLs: If the application has been tampered with and the AppInstaller URI isn't available, Windows will use these URLs to retrieve the latest version of the package and repair the broken installation.

The first feature is enabled through the **`UpdateUri`** section, which looks like this:

```
<UpdateUris>
        <UpdateUri>https://www.contosobackup.com/
           ContosoExpenses.AppInstaller</UpdateUri>
       <UpdateUri>\\MyNetworkShare\\ContosoExpenses
         .AppInstaller</UpdateUri>
</UpdateUris>
```

The second one is enabled by the **`RepairUris`** section, which is defined as follows:

```
<RepairUris>
        <RepairUri>https://www.contosobackup.com/
          ContosoExpenses.AppInstaller</RepairUri>
        <RepairUri>\\MyNetworkShare\\ContosoExpenses.
          AppInstaller </RepairUri>
</RepairUris>
```

We have completed the exploration of another way to distribute our Windows applications. This is via a website or a network share, which is a scenario made easier by MSIX and AppInstaller. Now, let's see the last option we're going to discuss in this chapter: Windows Package Manager.

Publishing your application to the Windows Package Manager repository

Installing applications can be a long and tedious process, especially if you're building a new machine and you have to reinstall all the tools you need for your work and personal file. What if you could somehow automate this process? What if you can make it faster rather than having to pick the applications you need one by one? These are the reasons that led Microsoft to create a new utility called **Windows Package Manager** (the short name is **`winget`**), which is inspired by other popular tools such as **Advanced Package Tool** (**APT**) and Chocolatey.

NOTE

Windows Package Manager is deployed through the AppInstaller application, which is built inside Windows. As such, you will find this utility

already available on every Windows installation starting from Windows 10 1709. Preliminary versions of the new releases are available through the official GitHub repository at **https://github.com/microsoft/winget-cli/releases**.

Windows Package Manager is a command-line tool that you can use to quickly install and upgrade applications on your machine from a variety of sources, such as websites and Microsoft Store. The tool is backed up by a repository, which is hosted on GitHub at **https://github.com/microsoft/winget-pkgs**. It contains a manifest for each available application. The manifest is a YAML file that describes the application, including the URL where it can be downloaded from. For example, this is an excerpt of the manifest of Microsoft Edge—the popular Microsoft browser:

```
# yaml-language-server: $schema=https://aka.ms/winget-
  manifest.singleton.1.0.0.schema.json
PackageIdentifier: Microsoft.Edge
Publisher: Microsoft
PackageName: Microsoft Edge
Author: Microsoft
ShortDescription: World-class performance with more
  privacy, more productivity, and more value while you
    browse.
Moniker: msedge
Tags:
…
MinimumOSVersion: 10.0.0.0
PackageVersion: 86.0.622.38
InstallerType: msi
Installers:
- Architecture: x64
  InstallerUrl: https://msedge.sf.dl.delivery.mp
    .microsoft.com/filestreamingservice/files/53a5f508-
       44da-4f9d-85d9-312fb8f92f4b/MicrosoftEdge
```

```
          EnterpriseX64.msi
    InstallerSha256: D49104F182675701423CB774
      CD2120B3A3FF63565661E4364D1169F64B9019EC
    Scope: machine
  PackageLocale: en-US
  ManifestType: singleton
  ManifestVersion: 1.0.0
```

The key information elements in the manifest are as follows:

- **`PackageIdentifier`**: This is the unique ID that we can use to install/upgrade/remove the application.
- **`InstallerUrl`**: This is the URL where the installer is available. In the preceding example, it's an MSI installer, but it could have also been an EXE package or an MSIX package.

If you want to install this application, you just need to open a Command Prompt or Windows Terminal and type in the following command:

```
winget install Microsoft.Edge
```

Windows Package Manager will display a progress bar, and it will let you know when the application has been installed. Thanks to **`winget`**, you can manage the full life cycle of the application. For example, you can use the following command to update Edge:

```
winget upgrade -q Microsoft.Edge
```

Also, you can use **`winget`** to remove an application by invoking the following command:

```
winget uninstall Microsoft.Edge
```

As a user, the whole **`winget`** backend is completely transparent. You don't have to know how and where manifests are stored. In fact, you can directly search whether an application is in the catalog with the **`search`** command, as shown in the following example:

```
winget search edge
```

The tool will give you a list of packages that matches the criteria and their identifiers so that you can install the one you're looking for, as shown in the following screenshot:

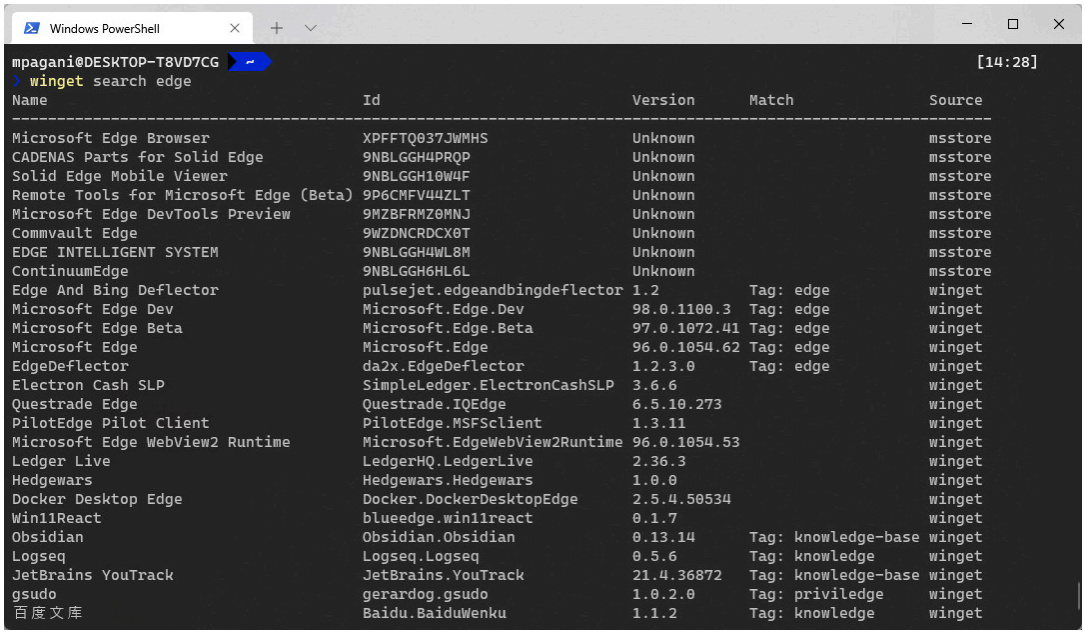

Figure 11.14 – The list of applications returned by a search in Windows Package Manager

As a developer, making your application available through Windows Package Manager is a worthwhile investment since the following can occur:

- You can make the life of users who want to automate the installation of applications much easier.
- Soon, Microsoft is going to enable the possibility of hosting your own private Windows Package Manager repository, which you'll be able to connect to enterprise tools such as Microsoft Endpoint Manager. This scenario will enable enterprises to manage their internal catalog of applications more efficiently since they won't be forced to host the various installers and packages internally anymore. Instead, they will just need to host the manifests. This solution is going to replace the Microsoft Store for Business as the official Microsoft solution to host an internal catalog of applications for enterprises. Therefore, by submitting your manifest, you will

make the lives of companies interested in acquiring your application easier.

In the next section, let's see how we can create a manifest.

Creating a manifest for Windows Package Manager

As mentioned earlier, a manifest is just a YAML file that contains the metadata of the application. As such, you could create it manually with any text editor by following the schema described at **https://docs.microsoft.com/en-us/windows/package-manager/package/manifest**.

However, Microsoft provides a command-line tool that you can use to generate and update manifests more easily. The tool is called **WinGetCreate**, and guess what? It's available through Windows Package Manager. To install it, just open a Terminal and run the following command:

```
winget install wingetcreate
```

Once the installation is complete, you can start the creation of a new manifest by calling the following command:

```
wingetcreate new
```

A wizard will guide you through the flow so that you can specify the following information:

1. The first information you must provide is the URL of your installer or MSIX package. The tool will download it and parse it, to try to automatically extract as much metadata as possible.
2. The package identifier must be unique. The tool will propose you one based on the publisher and the name of the application.
3. The version number.
4. The default language of the application (for example, **`en-US`**).

5. The publisher's name.
6. The package name.
7. The package license.
8. A short package description.

The tool will automatically generate the manifest file in a path that follows the same rule adopted by the official repository, which is as follows:

```
manifests\<first letter of the publisher>\<name of the
  publisher>\<name of the application>\<version number>
```

For example, if you explore the official repository on GitHub, you will find that the Microsoft Edge manifest is stored in the following path:

```
Manifests\m\Microsoft\Edge\86.0.622.38\Microsoft.Edge.y
aml
```

Once the manifest has been generated, the tool will give you the option to submit it to the Windows Package Manager repository. This is so that your application can be made available in the catalog. If you have created the manifest manually, you can validate it before submitting it, to make sure you haven't made any mistakes, by using the following command:

```
winget validate <path of the YAML file>
```

Regardless of the way you have created the manifest, to submit it, the **`wingetcreate`** tool will first ask you to log in to GitHub with your account.

The mechanism behind the manifest submission is the **`Pull Request`**. Once the manifest has been submitted, you will find a new pull request in the Windows Package Manager Community repository, which will add the manifest you have just created to the catalog. The pull request will trigger a validation process, which will verify that the metadata is correct, the installer URL is valid, the binaries are safe, and more. Once the pull request has been approved, the new manifest will

be merged inside the main repository, and you'll be able to start using the **`winget install`** command to install your application.

The **`wincreate`** tool can also be used to create an update for an existing manifest and submit it to the repository, as shown in the following example:

```
wingetcreate update MatteoPagani.ContosoExpenses -v
  1.0.201.0 -u https://www.contoso.com/ContosoExpenses
    102010.msix
```

Unlike the create option, the updated one isn't interactive, which makes it a good fit to be integrated inside a CI/CD pipeline. We'll learn more about this scenario in ***Chapter 12***, *Enabling CI/CD for Your Windows Applications.*

Summary

Finding the right technology and platform to distribute your application is essential for building a successful product. However, once the application has been installed, your work as a developer doesn't stop there: you must manage the licensing, the monetization, the feedback, and the automatic updates. All of these tasks are equally important to continuously deliver value to your users.

In this chapter, we have explored some of the technologies (such as MSIX) and the platforms (such as Microsoft Store, sideloading with AppInstaller and Windows Package Manager) that you can use to plan a successful deployment and, at the same time, simplify many of the challenges that you face as a developer. Examples of this include setting up automatic updates with AppInstaller, delivering a clean and reliable clean installation and uninstallation experience with MSIX, and more.

This chapter serves as a foundation for the next and concluding chapter: we're going to see how, thanks to the technologies we have learned, we'll be able to automate building and deploying our Windows desktop application, thanks to the adoption of DevOps best practices.

Questions

1. Windows Package Manager is an alternative distribution channel if you don't want to use Microsoft Store or AppInstaller. Is this true or false?
2. To enable automatic updates with AppInstaller, you don't need to change the code of your application. Is this true or false?
3. Microsoft Store is the best fit to deploy applications in an enterprise environment. Is this true or false?